

# Don't Let the Model Write the YAML: Deterministic, Minimal-Diff GitOps Remediation from LLM-Proposed Field Changes

Pruthvi Davineni  
Independent Researcher  
pruthvi.davineni@gmail.com

**Abstract**—LLM agents increasingly diagnose production incidents and propose remediations. In a GitOps workflow, applying such a fix means editing a version-controlled configuration file. The obvious implementation, having the model author the edited file or a diff of it, is the one practitioners reach for first. We evaluate that choice on real Kubernetes manifests and find that which failure you get depends on how the model is asked and how capable it is, but that no text-generation strategy is suitable for unattended automation. Asking a model for a unified diff is unsafe: under strict patching almost none apply, but that is an artifact: a tolerant patch tool (GNU patch --fuzz) applies 96% of the same diffs, yet silently misapplies ~1 in 7 (14–20%), landing the change at the wrong place with no error signal. Asking a model to rewrite the whole file is capability-dependent: a small model reflows and corrupts the file, while a frontier model is usually correct, but is non-deterministic (the same request silently drops a field or edits a neighbor on a fraction of tasks) and must regenerate the entire file, so its cost is  $O(\text{file size})$  per edit no matter how small the change.

We present an alternative that separates the semantic decision (which resource, which field, which value) from the syntactic act of editing the file. The agent emits only a structured field-change intent; a deterministic pipeline indexes manifests by Kubernetes (kind, name), locates the target scalar's exact character span using the YAML parser's node position marks, and replaces only that span in the raw text. Because the file is never re-serialized, the diff is minimal by construction (a single scalar), formatting and comments are preserved, and the edit is correct and deterministic independent of the model: the same result whether driven by a small model or a frontier one, at  $O(1)$  generation cost (a fixed field-change intent, independent of file size) rather than the  $O(\text{file size})$  cost of regenerating the file. The contribution is not the span-edit primitive (structured editing is well known) but the pairing of an LLM-proposed intent with a deterministic, fail-closed application contract for GitOps. We implement it in KubeAstra (Apache-2.0) and release the benchmark (corpus, oracle, harness). Our claim is scoped to faithful application of a known change; whether the change is the right one is left to human PR review.

---

**Index Terms**—GitOps, LLM agents, configuration management, Kubernetes, automated program repair, deterministic editing, YAML

## 1. Introduction

LLM agents now propose infrastructure fixes, and GitOps (Argo CD, Flux) [10, 11, 12] makes the Git repository the source of truth: to “apply the fix” is to edit a manifest

and open a pull request. The naïve path, where the model writes the new file or a diff, is the one everyone reaches for, and, as we show, the one that fails or hides a latent hazard.

This paper makes a narrow, testable claim about the application step, deliberately separated from the (harder, human-reviewed) question of whether the proposed fix is correct. Given a known, correct semantic change (“api-gateway needs spec.replicas: 5”), how should the system turn it into bytes on disk? We compare three text-generation strategies against a deterministic one, across a small and a frontier model, on real manifests.

Findings (previewed):

1. Diff generation is unsafe for unattended use, and the reason is subtle (§6.7). Under strict application almost no model diffs apply (2.7% for the frontier model), but that is a lower bound: most rejections are wrong @@ line numbers, not wrong content, and a tolerant tool (GNU patch) applies 96%. The real problem is that 14–20% of the applied diffs are silently misapplied (wrong result, no error), which in an automated commit-and-ship pipeline is worse than an honest rejection. A validate-and-retry loop does not fix it (the retry re-authors a diff with the same failure mode).
2. Full-file rewrite is capability-dependent and unsafe for automation. A small model reflows the file wholesale (2.4% correct; 97.6% of outputs alter an unrelated line; mean diff 921 lines). A frontier model rewrites faithfully most of the time (97.6% correct, near-minimal diff), but is non-deterministic: 7.2% of tasks (6/83) yield a correct edit on some seeds and a corrupted one (dropped field, collateral change) on others. For a change that will be committed and shipped, non-deterministic silent corruption is disqualifying.
3. Deterministic span-editing removes the model from byte-level editing entirely. It is 100% correct, 0% collateral, perfectly minimal (one scalar), and identical across both models and all 415 runs, because the model never authors file content.

Thesis. The model is good at the semantic question and

unreliable at the syntactic one. Constrain the model to a structured intent; apply it deterministically. Correctness then follows from the algorithm rather than the model’s capability, and is free of the cost, latency, and variance of regenerating file text.

Contributions:

- C1. A design principle for safe LLM-driven configuration change: separate the semantic intent (resource, field, value) from the deterministic application to the file, with the resulting minimal-diff, no-fabrication, and determinism guarantees stated and proven by construction.
- C2. A concrete mechanism, YAML node-mark span-editing: replace exactly the target scalar in the raw text using the parser’s character-position marks, preserving comments, quoting, and key order without re-serialization; with named-list resolution (containers/env by name, not index) and explicit refusal on ambiguous or absent targets.
- C3. An open-source implementation in KubeAstra (Apache-2.0): repository indexing without a git binary or disk writes, a two-phase preview→open protocol, a CI-enforced no-auto-merge invariant, and a Kustomize patch-append fallback.
- C4. A reproducible benchmark and evaluation on real manifests across two models and five seeds, measuring correctness, collateral change, fabrication, format preservation, diff minimality, and determinism, the last being where the deterministic approach separates decisively from a frontier model that otherwise looks competitive.

## 2. Background & Motivation

GitOps. Desired state lives in Git; a controller reconciles the cluster to it; changes flow through reviewed PRs, giving audit, rollback, and a human gate [10].

LLM agents for operations. ReAct-style agents [1] have moved from suggesting a command to changing the configuration. The remediation step is where an agent stops advising and starts editing production source.

Why YAML is hostile to “just re-dump it”. Kubernetes YAML is whitespace-significant, comment-bearing, supports anchors/aliases and multi-document files, and encodes operator intent in comments and key ordering. Parsing to an object model and re-serializing is lossy: it reflows, re-quotes, reorders, and strips comments. Any approach that round-trips the file through a serializer forfeits fidelity before the model is even involved.

The gap. Existing agent tooling either (a) applies changes imperatively to the live cluster, bypassing GitOps, or (b) lets the model author the file or diff, which we show is unreliable. Neither yields a reviewable, minimal, guaranteed-faithful GitOps change.

## 3. Problem Statement & Design Goals

Setting / threat model. A semi-autonomous agent proposes a change; a human reviews the PR; the agent is

not trusted to write file bytes or to merge. Even a well-intentioned model is an untrusted diff author.

Design goals (testable). - G1 Minimal diff: the change touches only the intended scalar; diff size is bounded by the edit, not the file. - G2 No fabrication / no collateral: every byte outside the target is unchanged. - G3 Fidelity: comments, quoting style, key order preserved. - G4 Determinism: identical intent + repo state  $\Rightarrow$  identical diff. - G5 Fail-closed: ambiguous or absent target  $\Rightarrow$  refuse, never guess. - G6 Reviewability: output is a PR; a human merges; the agent cannot.

Scope (v1). Scalar field changes (replicas, container image tag, resource requests/limits, env values) on Deployment/StatefulSet-style resources, in plain-YAML and Kustomize repositories. Non-goals (Helm/Argo value indirection, structural edits) are stated in §7.

## 4. Approach

4.1 The field-change intent (the interface — C1): The agent emits a structured intent, not file text: `FieldChange{ kind, name, namespace?, field_path, new_value, reason }`. `field_path` uses map keys with named list segments (a container by its name, an env var by its name), never positional indices. This is the crux: the model’s entire contribution is which scalar and what value.

4.2 Repository indexing without git or disk: Fetch the repo as a gzip tarball via the hosting API; extract in memory; parse every \*.y\*ml with a safe multi-document loader; skip files that don’t parse (Go-templated Helm charts) rather than failing. Index each document by (kind, name); namespace disambiguates rather than keys (Kustomize sets it centrally). No git binary, no token on disk, no manifest cache.

4.3 Node-mark span location (the mechanism — C2): Span-editing via a parser’s position marks is a known structured-editing technique (IDE refactorings, tree-sitter tooling [7], and semantic patchers [3] all edit located spans); our contribution is not the primitive but its use under an LLM-intent + fail-closed refusal contract for GitOps remediation. Compose the target document into a node tree whose scalar nodes carry start/end character-position marks. Walk the tree along `field_path`, resolving named-list segments by matching the item’s name child, and return the terminal scalar’s [start, end) character span.

Ambiguity (G5): 0 matches  $\Rightarrow$  refuse (hinting if Helm/Argo markers are present);  $>1 \Rightarrow$  prefer the environment overlay, else refuse and list candidates; field absent  $\Rightarrow$  refuse.

4.4 Span replacement (why the guarantees hold — C1): `edited = raw[start] + requote(new_value) + raw[end:]`, with no re-serialization. `requote` reproduces the original scalar’s quoting style so a quoted numeric stays quoted (avoiding a type coercion and a larger diff). - G1/G2: every byte outside [start,end) is unchanged by construction  $\Rightarrow$  minimal diff, zero collateral, zero fabrication. -

## Node-mark span-editing: the model names the scalar, code edits the bytes

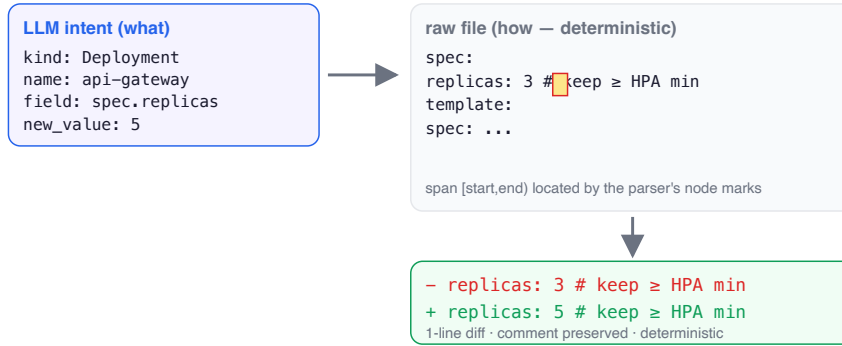


Figure 1: node-mark span-editing — the model names the scalar; deterministic code locates its byte span and replaces only it, yielding a 1-line diff with the comment preserved.

G3: comments/quoting/order live outside the span  $\Rightarrow$  preserved. - G4: pure function of (raw text, span, value)  $\Rightarrow$  deterministic.

Lemma (minimality). Let the located span be  $[s,e)$  and  $r = \text{requote}(v)$ ; the output is `raw[:s] + r + raw[e:]`. Every byte outside  $[s,e)$  is copied verbatim, so the line-diff against raw touches only the line(s) spanning  $[s,e)$ . For the in-scope field types a scalar span lies within one line, so the diff is exactly that one line, and its edit distance equals the edit distance between the old and new scalar representations, the minimum possible for the change. Collateral and fabrication are identically zero. (The guarantee is over the byte-level edit; it does not assert the change is semantically correct; that is the intent’s responsibility, §7.)

4.5 Kustomize patch-append fallback: When the resource is generated (no checked-in source line) but the repo is Kustomize with a known overlay, append a strategic-merge patch and its `kustomization.yaml` entry: one commit, still reviewable, the only case that adds a file.

4.6 From edit to pull request (C3): Two-phase protocol: preview returns the real diff and a single-use token (nothing pushed); open pushes branch+commit+PR via the hosting Git Data API (`blob` $\rightarrow$ `tree` $\rightarrow$ `commit` $\rightarrow$ `ref` $\rightarrow$ `pull`). No-merge invariant: no code path calls a merge endpoint, enforced by a CI test. The human always merges.

## 5. Implementation

KubeAstra (Apache-2.0). The GitOps component is ~520 lines of Python across eight modules; the safety-critical core, the locator (§4.3) and the span editor (§4.4), is ~150 lines. Zero new native dependencies: an HTTP client and the stdlib-level YAML parser only; heavier libraries were avoided so the signed, notarized desktop distribution stays simple. 36 dedicated tests cover the locator, round-trip invariants, quote preservation, and the CI-enforced no-merge guard. The component runs

identically in server and desktop modes; the token comes from an environment secret or the OS keychain.

(The mechanism’s small size is itself a result: correctness comes from ~150 lines of deterministic code, not from a capable model.)

## 6. Evaluation

### 6.1 Research questions:

- RQ1 (correctness): Given a known change, does each strategy produce exactly that change?
- RQ2 (safety, collateral/fabrication): Does it alter or invent anything else?
- RQ3 (fidelity & minimality): Are comments/formatting preserved and the diff minimal?
- RQ4 (determinism): Does the same request produce the same edit across seeds?
- RQ5 (capability dependence): How do the answers change between a small and a frontier model?
- RQ6 (fail-closed, G5): On adversarial targets (absent, ambiguous, templated, non-scalar, alias), does the locator refuse rather than guess?

6.2 Corpus: 83 field-change tasks generated from two Apache-2.0 sources, SHA-pinned with recorded provenance: the Online Boutique manifests (a single ~9.3K-token, multi-resource file, the realistic hard case) and a Kustomize helloWorld base/overlay. Each task is (manifest, kind, name, field\_path, old\_value, new\_value) with a deterministic new value; field types span replicas, image tag, resource requests/limits, and env values.

We are explicit about the corpus’s limits. Because the multi-resource file yields many edit sites, ~80 of the 83 tasks are edits within that single file; the corpus thus exercises many edit sites but few distinct files. Reported rates should be read as edit-site rates within a small file set, not population estimates; we therefore round to two decimals and treat single-file-dependent findings (notably the flaky-6 determinism result) as indicative, to be confirmed on the  $\geq 250$ -task, multi-repo corpus planned

for the peer-reviewed version (§7). This preprint’s claims rest on effects that are large and consistent across all 415 runs (diffs failing under strict patching; span-edit’s by-construction determinism), not on the third decimal of any rate.

### 6.3 Systems:

- SUT: the deterministic span-edit pipeline (§4), given the structured intent.
- B1 (full-file): the model rewrites the entire edited file.
- B2 (unified-diff): the model emits a unified diff, applied strictly (any context mismatch = failure, a real, measured weakness, not an artifact).
- B3 (diff + retry): B2 plus validate-and-retry (re-prompt with the apply/parse error, up to 2 retries).

All baselines receive the same engineered prompt with a natural-language change description and a worked one-shot example (a fair, not straw-man, comparison; prompts in Appendix A). Two models: gemini-3.7-flash (small) and claude-sonnet-5 (frontier). 5 seeds, 415 runs per system per model. Correctness is judged by an automated parse-and-compare oracle: it parses both manifests, confirms the target field equals new\_value, and flags any other changed field as collateral and any added structure as fabrication, so a semantically-correct-but-reflowed output still scores correct on RQ1 while failing fidelity, and baselines are never straw-manned on formatting alone.

6.4 Results: RQ1/RQ2: full-file is capability-split; diffs are unsafe under strict and lenient application. Under strict application unified diffs almost never apply (Table 1: 3.6% flash / 1.2% Sonnet correct), but strict is a lower bound; §6.7 gives the full applier spectrum, where a tolerant tool applies 96% of the same diffs yet silently misapplies 14–20%. So the diff baselines are unsafe either way (rejected, or applied-but-wrong-with-no-signal), not merely inaccurate. Full-file rewrite splits by capability: flash reflows the whole file (2.4% correct, 97.6% collateral, 921-line diffs), while Sonnet rewrites faithfully 97.6% of the time with near-minimal diffs. In absolute terms the entire frontier-B1-vs-SUT correctness gap is 10 failed runs out of 415 (405 vs 415 correct); it is small, and we do not rest the case on it. Span-edit’s decisive advantages are determinism (RQ4, where those 10 failures fall on 6 tasks that flip across seeds),  $O(1)$  cost (§6.6), and capability-independence (RQ5), not raw accuracy.

The result is not driven by a few easy or hard edits.

SUT is 1.00 for every field type. B2’s diffs fail near-uniformly (0.00–0.02) whatever is changed: the failure is diff generation itself, not any particular edit. Flash’s B1 fails across the board (it reflows the whole file regardless of target). The replicas row ( $n = 10$ , two tasks) is small and noisy; it is the only place a diff baseline reaches double digits, consistent with replicas being a single self-contained numeric line.

RQ4: The decisive axis is determinism. SUT is correct on all 5 seeds for all 83 tasks (0 flaky across 415 runs). Sonnet’s B1, despite 97.6% aggregate correctness, is flaky

on 6/83 tasks: the identical request produces a correct edit on some seeds and a corrupted one (5 tasks drop the target field on some seeds; some make a collateral edit) on others. In an automated GitOps loop that commits and ships, a strategy that is usually right but silently wrong on retry cannot be trusted; determinism, not average accuracy, is the property that matters.

RQ5: Capability dependence, and its removal. The only strategy whose result does not move between a \$0.10 model and a frontier model is the deterministic one. Full-file quality tracks capability (0.024  $\rightarrow$  0.976); diff quality is poor at both. Span-editing is 1.000/deterministic regardless, because the model never authors bytes.

6.5 A note on methodology (max\_tokens): An early run capped output at 4,000 tokens; because B1 must re-emit the entire  $\sim 9.3$ K-token file, this silently truncated the frontier model’s B1 output, making it appear to fail (2.4%) when the true clean figure is 97.6%. We report all B1 numbers at a cap exceeding the largest file. Diff-based systems (B2/B3) emit small outputs and are unaffected. We flag this because it is exactly the kind of harness artifact that can invert a headline: a truncated B1 baseline would have overstated our result and been trivially refuted by a reviewer re-running B1 with a larger cap.

6.6 Cost (RQ, generation tokens per edit): Correctness is not the only axis that matters for automation; the strategies differ by two orders of magnitude in generation cost, and, critically, the frontier strategy that is most correct (B1) is also the one whose cost grows with file size. Table 4 reports model output tokens per edit and per correct edit; span-editing does not invoke the model for the edit at all (the intent is emitted once by the diagnosis step, and application is deterministic). Values are approximate, derived from the measured aggregate output of the Sonnet 5 run and the structural fact that B1 re-emits the whole file.

Empirically, the full Sonnet 5 run (83 tasks  $\times$  5 seeds, all three baselines) cost \$34.69, of which B1’s whole-file regeneration dominated the output tokens. The takeaway is not the absolute figure but the scaling: the one text strategy that is reliably correct (frontier B1) is also the one whose per-edit cost is  $O(\text{file size})$ , while span-editing’s generation cost is a fixed, small intent regardless of how large the manifest is.

6.7 Strict vs. lenient diff application — the diff story revised: The strict apply-rate is a lower bound, and a fair evaluation must ask what a real, tolerant patch tool would do. We captured all 415 Sonnet 5 B2 diffs and re-applied each under progressively more lenient appliers, scoring not just whether it applied but whether it applied correctly (Table 5). The distinction between applied and correct is the whole point. (The strict rate here, 2.7%, comes from this study’s own capture of the 415 diffs and differs slightly from Table 1’s main-run B2 Sonnet figure, 1.2%; both confirm that strict application rejects nearly

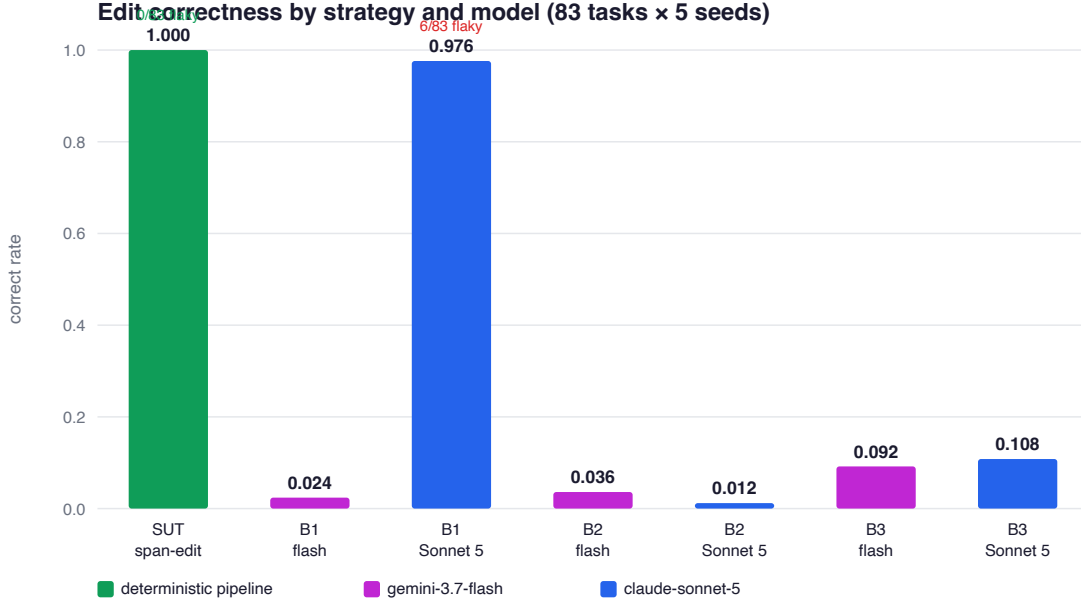


Figure 2: edit correctness by strategy and model. SUT (span-edit) is 1.0 and deterministic (0/83 flaky) regardless of model; the frontier model’s full-file rewrite (B1) is high but non-deterministic (6/83 flaky); diff-based baselines fail at both tiers.

Table 1 — Clean results (max\_tokens=16000, 5 seeds, 415 runs/system).

| Model    | System          | Correct | Refused | Collateral | Fabric. | Format kept | Mean diff |
|----------|-----------------|---------|---------|------------|---------|-------------|-----------|
| —        | SUT — span-edit | 1.000   | 0.000   | 0.000      | 0.000   | 1.000       | 2         |
| Flash    | B1 full-file    | 0.024   | 0.000   | 0.976      | 0.000   | 0.024       | 921.3     |
| Flash    | B2 unified-diff | 0.036   | 0.964   | 0.000      | 0.000   | 0.036       | 2         |
| Flash    | B3 diff+retry   | 0.092   | 0.908   | 0.000      | 0.000   | 0.092       | 2         |
| Sonnet 5 | B1 full-file    | 0.976   | 0.000   | 0.024      | 0.024   | 0.969       | 2.04      |
| Sonnet 5 | B2 unified-diff | 0.012   | 0.988   | 0.000      | 0.000   | 0.012       | 2         |
| Sonnet 5 | B3 diff+retry   | 0.108   | 0.892   | 0.000      | 0.000   | 0.108       | 2         |

SUT is model-independent: it never invokes the model for the edit, so its row is identical for both.

Table 2 — Determinism (RQ4): tasks correct on all 5 seeds vs. flaky (mixed).

| System                  | All-5-correct | Flaky (mixed across seeds)  |
|-------------------------|---------------|-----------------------------|
| SUT — span-edit         | 83 / 83       | 0 / 83 <sup>1</sup>         |
| B1 full-file (Sonnet 5) | 77 / 83       | 6 / 83                      |
| B1 full-file (Flash)    | 2 / 83        | 0 / 83 (consistently wrong) |

<sup>1</sup> SUT’s determinism is by construction (it makes no model call for the edit), so this row confirms rather than discovers it; the empirical content is the baselines’ variance.

all model diffs, and per §6.2 the claim rests on that large effect, not the exact figure.)

This revises the naive reading of B2 in two ways, and we state both plainly. 1. Strict application badly understates diff applicability: we retract “model diffs do not apply.” Most rejections are wrong @@ line numbers, not wrong content: an applier that locates each hunk by content (ignoring the line numbers) lifts correctness from 2.7% to 67.5% with zero misapplication, and GNU patch applies 96.4%. The blanket claim is false. 2. But lenient application trades rejection for silent misapplication. Under the real production tool (patch --fuzz), 14.0% of applied diffs are wrong (landed at the wrong location or corrupted

a neighbor, with no error signal), rising to 20.2% when whitespace is ignored (indentation is semantic in YAML). The safe lenient applier (offset-tolerant, content-unique match) refuses those, capping at 67.5%; the aggressive tool takes them and misapplies ~1 in 7.

For an unattended GitOps loop that commits and ships, the disqualifying failure mode is not rejection but silent corruption: a strict applier rejects most diffs; a lenient one silently misapplies 14–20% of them. Span-editing has neither mode: it is correct by construction (0% misapplied, Table 1).

6.8 Fail-closed refusal (G5), measured: G5 (refuse rather than guess on an ambiguous, absent, or unresolvable

Table 3 — Correct rate by field type (Sonnet 5; plus flash B1),  $n = \text{tasks} \times 5 \text{ seeds}$ .

| Field type     | n   | SUT  | B1 (Sonnet) | B2 (Sonnet) | B3 (Sonnet) | B1 (flash) |
|----------------|-----|------|-------------|-------------|-------------|------------|
| env value      | 100 | 1.00 | 0.96        | 0.01        | 0.10        | 0.00       |
| image tag      | 65  | 1.00 | 1.00        | 0.00        | 0.11        | 0.08       |
| limit.memory   | 60  | 1.00 | 1.00        | 0.00        | 0.13        | 0.00       |
| limit.cpu      | 60  | 1.00 | 1.00        | 0.00        | 0.10        | 0.00       |
| request.memory | 60  | 1.00 | 0.98        | 0.00        | 0.12        | 0.00       |
| request.cpu    | 60  | 1.00 | 0.98        | 0.02        | 0.03        | 0.00       |
| replicas       | 10  | 1.00 | 0.60        | 0.30        | 0.50        | 0.50       |

Table 4 — Generation cost per edit (Sonnet 5; approximate).

| Approach        | Output tok / edit       | Correct rate | Output tok / correct edit | Cost in file size     |
|-----------------|-------------------------|--------------|---------------------------|-----------------------|
| SUT — span-edit | $\sim 50\text{--}150^1$ | 1.00         | $\sim 50\text{--}150$     | $O(1)$                |
| B1 full-file    | $\sim 9,300^2$          | 0.976        | $\sim 9,500$              | $O(\text{file size})$ |
| B2 unified-diff | $\sim 200$              | 0.012        | $\sim 16,700$             | $O(1)$                |
| B3 diff+retry   | $\sim 560$              | 0.108        | $\sim 5,200$              | $O(1)$                |

<sup>1</sup> Not literally zero: the model emits the structured FieldChange intent ( $\sim 50\text{--}150$  tokens), which the diagnosis step must produce under any strategy; application is deterministic and model-free — a fixed cost, independent of file size. <sup>2</sup>  $\approx$  the file’s token length — B1 regenerates the entire file to change one scalar.

Table 5 — the same 415 model diffs under different appliers.

| Applier                          | Applied | Correct | Misapplied (applied but wrong) |
|----------------------------------|---------|---------|--------------------------------|
| strict (context-exact)           | 0.027   | 0.027   | 0.000                          |
| offset-tolerant (YAML-safe)      | 0.675   | 0.675   | 0.000                          |
| whitespace-insensitive           | 0.675   | 0.670   | 0.005                          |
| GNU patch -fuzz=3                | 0.964   | 0.824   | 0.140                          |
| GNU patch -fuzz=3 -l (ignore ws) | 0.964   | 0.761   | 0.202                          |

target) is a safety claim, so we test it on a small labeled adversarial stratum: controls that must edit, six categories that must refuse (absent resource, absent field, named-list/container miss, duplicate (kind,name), Go-templated/unparseable file, non-scalar target), and a YAML-alias probe.

Results: refusal precision 1.00 (no resolvable case is ever refused), control coverage 1.00 (every resolvable target edited correctly), and refusal recall 0.889. The fail-closed contract fires on every adversarial category except one: a field defined through a YAML alias resolves to its anchor node, so the locator edits the anchor’s line instead of refusing (yielding a dangling alias). We report this leak rather than hide it: it is a genuine gap, and the fix (detect anchor/alias nodes and refuse) is future work; until then anchors/aliases are out of scope (§7). The takeaway: on absent/ambiguous/unresolvable targets the contract holds with perfect precision, and aliases are the one known hole.

## 7. Discussion & Limitations

- We are not claiming “LLMs cannot edit YAML.” A frontier model rewrites files faithfully most of the time. We claim that (a) diff authoring is unreliable at every tier, and (b) full-file authoring is non-deterministic and unbounded-cost, and that a deterministic span-edit dominates on the properties automation needs (correctness, determinism, minimality, cost), at any capability level.

- The model can still choose the wrong intent. Our guarantees are about faithful application, not about the fix being right; that is what human PR review is for. We evaluate application in isolation by giving every system the same known change.
- Scalar-only, v1. Structural edits (adding a container/block) are future work; the intent schema addresses existing scalars.
- Helm / Argo value indirection. A live Deployment may be generated from a values file at a different kind/path; v1 refuses with a clear message. A resolver is future work.
- GitHub-only application path (Git Data API); GitLab is mechanical. Locator requires a parseable file; templated files are skipped, not edited.
- Corpus size and diversity. 83 tasks but  $\sim 80$  within a single file (§6.2). The peer-reviewed version requires a corpus drawn from  $\geq 50$  distinct repositories (source diversity matters more than raw task count) spanning plain YAML, Kustomize, and Helm-rendered manifests, plus an adversarial/refusal stratum. The flaky-6 determinism result must be re-derived across independent codebases before it can be stated as a model property rather than single-file behavior.

Threats to validity (the ones a reviewer will raise first):

- Strict vs. lenient diff application: now measured (§6.7), no longer a caveat. We re-applied all 415

Sonnet B2 diffs under strict, a YAML-safe offset-tolerant applier, and GNU patch --fuzz. Strict is indeed a lower bound (2.7% correct → 67.5% under offset-tolerance → 96.4% applied under GNU patch), so we retract any claim that model diffs simply do not apply. The finding that replaces it is stronger and more honest: GNU patch silently misapplies 14–20% of the diffs it accepts: the failure mode that actually matters for unattended GitOps. (git apply was evaluated and excluded: it is stricter than patch on the headerless, no-context single-line hunks models emit, and would depress the baselines further and unfairly.)

- Fail-closed (G5) is now measured (§6.8), with one known gap. On an adversarial stratum the contract holds with precision 1.00 and recall 0.889 across six categories (absent resource/field, container miss, duplicate (kind,name), templated file, non-scalar target). The single leak is YAML aliases: the locator resolves an alias to its anchor node and edits the wrong line rather than refusing, so anchors/aliases are out of scope until it gains alias detection. Separately, the main 83-task corpus never needed refusal, so coverage/refusal on a large, diverse corpus remains part of the ≥50-repo future work.
- Oracle validity, audited. All rates depend on the automated parse-and-compare oracle, which is covered by unit tests including adversarial cases (a reflowed-but-correct output passes correctness and fails fidelity; injected collateral and dropped-field outputs are flagged). We also ran an independent manual audit of a stratified random sample (n = 30: 10 SUT edits, 10 patch --fuzz outputs the oracle called correct, 10 it called misapplied): the oracle’s verdict matched independent adjudication on all 30, including every misapplied case the 14–20% finding rests on. The audit is reproducible (oracle\_audit.py in the artifact) and makes the mechanism concrete: an identical cpu: 100m to 200m hunk applied at the same line is correct when that resource was the target and misapplied when an identical-context neighbor was, and the oracle separates the two by (kind, name).
- Two models is not a gradient. “Capability-dependent” is inferred from two points (one small, one frontier); a third model would establish monotonicity. The flaky tasks are also not yet characterized (why those six).

## 8. Related Work

- LLM agents and LLM-driven code editing. ReAct-style reasoning-and-acting agents [1] underpin most remediation agents. Benchmarks such as SWE-bench [2] evaluate whether models can edit a codebase to resolve an issue, and coding agents typically have the model author a patch or a search-and-replace edit, exactly the byte-level authoring whose failure modes we quantify. Position: for the bounded domain of

scalar config changes, we remove the model from byte-level editing entirely and give guarantees a generative patch cannot.

- Structured and semantic patching. Coccinelle’s semantic patch language [3] performs reliable, large-scale, structure-aware transformations of C code; tree-sitter [7] and lint autofixers such as Semgrep [8] apply AST- or rule-driven edits. Position: we adapt structured editing to LLM-intent-driven GitOps remediation, with explicit minimal-diff/no-fabrication guarantees and fail-closed refusal.
- Automated program repair. APR generates candidate patches and validates them against tests [4]. Contrast: we do not generate a patch; we faithfully apply a specified change, so correctness of application is guaranteed and only the intent is model-decided.
- LLMs for cloud operations / AIOps. LLMs are used for incident root-cause analysis and mitigation recommendation [5, 6] and surveyed for failure management [9]; these systems propose fixes but stop short of a guaranteed-faithful GitOps edit. Position: we address the application step they leave open.
- GitOps foundations. The OpenGitOps principles [10] and controllers such as Argo CD [11] and Flux [12] establish Git as the source of truth and the PR as the change unit, the setting our approach targets.

## References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR 2023. arXiv:2210.03629.
- [2] C.E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” ICLR 2024. arXiv:2310.06770.
- [3] Y. Padiou, J. Lawall, R. R. Hansen, G. Muller. “Documenting and Automating Collateral Evolutions in Linux Device Drivers.” EuroSys 2008.
- [4] M. Monperrus. “The Living Review on Automated Program Repair.” Technical report, HAL hal-01956501.
- [5] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, et al. “Automatic Root Cause Analysis via Large Language Models for Cloud Incidents.” ASPLOS 2024. arXiv:2305.15778.
- [6] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, S. Rajmohan. “Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models.” ICSE 2023. arXiv:2301.03797.
- [7] M. Brunsfeld et al. “Tree-sitter: An incremental parsing system for programming tools.” <https://tree-sitter.github.io>.
- [8] Semgrep. “Semgrep: Lightweight static analysis for many languages.” <https://semgrep.dev>.
- [9] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, et al. “A Survey of AIOps for Failure Management in the Era of Large Language Models.” arXiv:2406.11213.
- [10] CNCF OpenGitOps Working Group. “GitOps Principles v1.0.0.” <https://opengitops.dev>.
- [11] Argo Project. “Argo CD — Declarative GitOps CD for Kubernetes.” <https://argo-cd.readthedocs.io>.
- [12] Flux Authors. “Flux — GitOps for apps and infrastructure.” <https://fluxcd.io>.

## Back matter

- Artifact: the benchmark (corpus, oracle, harness, and the result data behind the tables) is released at [github.com/astraverse-io/kubeastra-bench](https://github.com/astraverse-io/kubeastra-bench);

the span-edit implementation is in KubeAstra ([github.com/astraverse-io/KubeAstra](https://github.com/astraverse-io/KubeAstra)). Both Apache-2.0.

- Ethics/impact: human-review + no-auto-merge; the technique reduces the risk of autonomous production change.

## Appendix A: Baseline prompts

All baselines receive the change as natural language plus a worked one-shot example, so they are a fair rather than straw-man comparison. The change description is generated from the structured intent (per field type: replicas, container image, env var value, or resources.{limits,requests}.{cpu,memory} of a named container), e.g. In Deployment/api, set spec.replicas to 5. The large manifest is placed at the END of the system message: prompt caching is a prefix match, so an identical system prefix lets the ~80 tasks that share a manifest, across all 5 seeds, read it from cache. Reproduced below with the manifest elided as <manifest>; the shared instruction is:

Apply EXACTLY the requested change and nothing else.

Change only the single field named. Preserve every comment, all formatting, quoting style, and key order. Do not reformat, re-quote, reorder, or touch any other field.

B1 (full-file) system message, then user message = the change description:

You edit Kubernetes YAML manifests. <shared instruction>  
Output ONLY the complete edited file, no explanation, no markdown code fences.

Example, change spec.replicas to 3:

FILE:

spec:

```
replicas: 2          # set by the on-call runbook
minReadySeconds: 10
```

OUTPUT:

spec:

```
replicas: 3          # set by the on-call runbook
minReadySeconds: 10
```

Apply the change described in the next message to this manifest, and output the complete edited file.

FILE:

<manifest>

B2 (unified diff) system message, then user message = the change description:

You edit Kubernetes YAML manifests by emitting a unified diff. <shared instruction> Output ONLY a unified diff with @@ hunk headers that applies cleanly to the file, no explanation, no markdown code fences.

Example, change spec.replicas to 3:

@@ -1,2 +1,2 @@

```
- replicas: 2          # set by the on-call runbook
+ replicas: 3          # set by the on-call runbook
```

Apply the change described in the next message to this manifest.

FILE:

<manifest>

B3 (diff + retry) is B2; on a rejected diff, the user message appends the apply/parse error and re-requests a

clean diff (up to 2 retries):

Your previous diff was rejected: <error>

Return a corrected unified diff that applies cleanly.

## Appendix B: Locator pseudocode

The locator (§4.3) is about 20 lines. Given the raw manifest text, the target document index, and the field path, it returns the target scalar's [start, end) character span, or None, which the pipeline turns into a fail-closed refusal:

```
function find_span(raw_text, doc_index, field_path):
    docs = yaml.compose_all(raw_text)    # node trees
    carrying position marks
    if doc_index out of range: return None
    node = docs[doc_index]
    for key in field_path:
        if node is a MappingNode:
            node = child whose key-scalar == key
        else if node is a SequenceNode:
            node = item whose 'name' child == key    # named
    list, not by index
    else:
        return None    # path descends into a
    scalar
    if node is None: return None    # key or 'name' absent
-> refuse
    if node is not a ScalarNode:
        return None    # target is a map/list ->
    refuse
    return Span(start = node.start_mark.index,
                end   = node.end_mark.index,
                old   = node.value)
```

The edit is then raw\_text[:start] + requote(new\_value) + raw\_text[end:]. Every return None is a G5 refusal: out-of-range document, missing map key, missing named-list item, or a non-scalar target. Ambiguity (duplicate (kind, name)) is caught upstream in the index; the alias leak is discussed in §6.8.

## Appendix C: Oracle (correctness checker)

The oracle judges an output on parsed structure, not on text, so a correct-but-restyled edit still counts as correct (fairness by construction). Given the original manifest, the system output, and the intent:

```
judge(original, output, intent):
    if output does not parse as YAML: incorrect (parses = false)
    locate the target document by (kind, name) at the SAME
    index in both files
    flat_o = flatten(output)    # leaf-path -> value, named-list
    aware
    flat_m = flatten(original)
    target = (doc_index,) + intent.field_path
    correct = (flat_o[target] == intent.new_value)    # target
    hit, right value/type
                and no other leaf added, removed, or changed    #
    zero collateral
```

Leaf paths through lists of uniquely-named mappings (containers, env, volumes) are keyed by name, not index, matching the locator. new\_value is compared in its intended parsed type (int for replicas, str for an image tag), so a string/int mismatch is a real difference and is reported. Two fidelity signals, comments\_preserved and



format\_preserved (only the target line changed), are computed from raw text as secondary metrics and never affect correct. The oracle is unit-tested against known-good and

known-bad pairs, including a reflowed-but-correct output (passes correctness, fails fidelity) and injected collateral and dropped-field outputs (flagged).